ENGR 102 Summer Bridge
Bridge Day 10 Student Workbook

Lectures 1-5 Recap, Direct Input, and Repetition Launch

Friday, July 17, 2026 • 50 points • tagged v5 successor

Name		**UIN**		**Section**	

Daily target and independence boundary

Connect input, state, Boolean decisions, and testing to a first complete while-loop program.

Allowed tools	input, int, float, arithmetic, comparisons, Boolean operators [[and]], [[or]], and [[not]], if/elif/else, assignment, print, and a first while loop after the quiz
Support supplied	Variable names, a floor-validity rule, the movement goal, and named boundary tests are supplied.
Student decides	The conversion, validation branch order, continuation condition, direction decision, and one-floor update.
Scaffold level	Planning frame supplied; the complete elevator loop is student-authored.

Evidence and scoring

Evidence	**Points**	**Secure work shows**
Integrated trace	14	intermediate state, condition results, exact output, and meaning
Diagnosis and repair	12	actual, intended, cause, repair, and a discriminating test
Complete program and tests	24	coherent executable logic, required output, assumptions, and defended tests

Task 1. Integrated trace - 14 points

Trace types, Boolean subexpressions, and branch order. Do not jump directly to the final line.

```
sample = "14"
offset = 5
value = int(sample) * 2 + offset
stable = 20 <= value <= 35 and value % 3 == 0

if not stable:
status = "review"
elif value >= 30:
status = "high"
else:
status = "normal"

print(value, stable, status)
```

Your task

1. Compute value and identify its type. Then evaluate each side of the [[and]] expression used to compute stable.
2. State which branch assigns status and explain why the earlier branch does not execute.
3. Write the exact printed output, including the Boolean capitalization.

Task 2. Diagnose and repair - 12 points

The requirement is: approval requires training, and it also requires either a temperature from 15 through 30 inclusive or a supervisor override.

```
trained = False
temperature = 20
supervisor = False

approved = trained and temperature >= 15 or temperature <= 30 or supervisor
print(approved)
```

Your task

1. Show the actual Boolean result for the supplied values and explain why it violates the requirement.
2. Write one corrected Boolean assignment and defend its grouping with a test in which trained is False.

Task 3. Construct a complete program - 24 points

Program specification

- Read current_floor and desired_floor as integers.
- Valid floors are 1 through 10 inclusive. Print invalid if either input is outside that interval.
- If the valid floors are equal, print already there.
- Otherwise, use a while loop that changes current_floor by exactly one floor per pass until it equals desired_floor.
- Print each newly reached floor inside the loop. Do not jump directly to the destination.

Required planning evidence

1. Write the invalid-input Boolean expression and the branch order that protects the loop.
2. Write the continuation condition and both possible one-floor updates before coding.

Program workspace

Task 3 continued. Test and defend - included in 24 points

Continue code if needed. Required tests, expected results, and brief defense

Bridge Day 10 VIP Evidence Handoff

STUDENT COPY • Contract 07a04826c975 • Accessible v5 successor

Why engineers use this page

Begin with the notebook's required read-convert-compute-format-print reconnect, then transfer the elevator loop's continuation, update, and termination evidence into three while-loop activities.

Decision boundary: Code is useful only when its stored state, visible result, assumptions, units, limits, and tests support a defensible engineering interpretation.

Construct readiness before independent work

New authoring tools: for loop, while loop, range call, loop state update, termination condition, list literal as collector, append call
Carried authoring tools: assignment, print call, arithmetic expression, modulo, input call, int conversion, f string, comparison expression, if elif else
Supplied-code exposure only: list index read
An exposure-only construct may be traced in complete supplied code, but the independent task will not require students to invent it before its authoring day.

Notebook cells and task constructs

Activity	Work	Stable cells	Required authoring constructs
18.19	Countdown To Multiple	`18-19-work / 18-19-transfer / 18-19-cold`	arithmetic expression, while loop
18.21	Cooldown Sequence	`18-21-work / 18-21-transfer / 18-21-cold`	append call, arithmetic expression, while loop
18.22	Rounds To Clear Queue	`18-22-work / 18-22-transfer / 18-22-cold`	arithmetic expression, f string, input call, int conversion, while loop

Readiness evidence

1. Write the five-part program shell, then for each mapped activity identify the input conversion, changing state, and continuation condition.

2. Explain in one sentence why the update moves the state toward termination.

Timing and help trigger

Record a first prediction before running. If you still lack an executable plan after about 8 focused minutes, use the linked ThinkCSPy refresher or the supplied model. If two attempts fail for the same named requirement, write the exact mismatch and ask a peer mentor or instructor a targeted question. Timing is diagnostic evidence, not a speed grade. **Start or elapsed minutes:** _____ **My help trigger:** _____

Engineering Evidence Record

Complete one record from a model, transfer, or Independent Program Studio build. Generic statements are not sufficient; use actual values, a real revision, and one concrete next test.

Evidence field	Record
Prediction	
Observed Result	
Revision Reason	
Engineering Meaning	
Units Or Not Applicable	
Assumption	
Model Limit	
Next Test	
Help Trigger	
Minutes Used	

Notebook and submission procedure

1. Attempt, predict, run, inspect, and revise before opening a reveal.
2. Complete the end-of-notebook Independent Program Studio without a reveal or copied solution. Only those visibly labeled Studio cells are scored for independent mastery.
3. Save or download the completed notebook as one `.ipynb` file.
4. Upload that one notebook to the matching Gradescope assignment. Do not export a `.py` file or create a ZIP.

Generated from the canonical VIP construct and evidence contract; workbook planning and executable notebook work remain distinct.